

Building VHDL Processors with Claude Code: From 16-Bit to 64-Bit FPU

George Teifel^{1*}, Siamak Tavakoli¹

¹ *Department of Electrical and Computer Engineering, University of New Mexico, Albuquerque, NM 87131, USA*

** Corresponding author: gteifel@unm.edu*

Abstract

Manual design of register-transfer level (RTL) hardware descriptions remains a labor-intensive process requiring deep expertise in both digital logic and synthesis tool chains. This paper presents a skill-based methodology for AI-assisted hardware description language (HDL) generation using Claude Code, an AI coding assistant. A custom skill was constructed by merging two community-contributed FPGA design skills and augmenting them with three capabilities: a simulation-first test-driven development (TDD) mandate, live documentation retrieval via Context7 MCP integration, and Vivado version-aware code generation. Using this skill, six complete RISC-V processor designs were generated in a single session: three in Verilog and three in VHDL, spanning 16-bit educational cores (RV32I subset), 64-bit integer processors (RV64I), and a 64-bit processor with a full IEEE 754 double-precision floating-point unit (RV64IFD). The most complex design implements 68 instructions across 28 FPU operations in a 5-stage pipeline, consuming 12,969 look-up tables (LUTs) and 18 DSP48E1 slices on a Xilinx Artix-7 XC7A35T. The simulation-first verification approach identified 10 floating-point bugs in the initial generation and drove a 48% reduction in LUT utilization, from approximately 25,000 to 12,969 LUTs. All designs were verified on a Digilent Basys 3 development board. The results suggest that domain-specific skill engineering can substantially improve the quality and completeness of AI-generated HDL.

I. Introduction

The design of digital processors at the register-transfer level demands simultaneous expertise in computer architecture, hardware description languages, synthesis tool chains, and verification methodology. Even experienced engineers require weeks to months to produce a verified processor design targeting a specific FPGA platform [3]. Recent advances in large language models (LLMs) have demonstrated promising capabilities in software code generation, but their application to hardware description languages remains largely exploratory, and prior results have been confined to small-scale designs [7], [8], [9], [11], [12].

The gap between LLM-generated HDL and practical processor design remains vast. Thakur et al. [7] benchmarked LLMs on small Verilog module tasks at DATE 2023, reporting that the best model achieved only 25.9% syntactically correct code and a mere 6.5% functional correctness rate — and these benchmarks targeted isolated modules such as counters, adders, and FIFOs, not complete processors. Pearce et al. [9] established early baselines at MLCAD 2020 with DAVE, which derived small Verilog modules from English descriptions but did not attempt system-level designs. Blocklove et al. [8] achieved the most ambitious prior result at MLCAD 2023 with Chip-Chat, generating an 8-bit accumulator-based microprocessor that was claimed as the world's first wholly AI-written HDL for tapeout — but it featured an 8-bit datapath, no pipeline, no floating-point unit, and a minimal instruction set. More recently, Sun et al. [11] used ChatGPT-3.5 to generate an RV32I processor in VHDL and Verilog, but the design was limited to 32-bit integer operations with no FPU and no dual-language parity. Zhao et al. [12] introduced ResBench with 56 FPGA benchmarks measuring resource efficiency of LLM-generated designs, but these remained module-level benchmarks rather than complete processors.

This paper presents a fundamentally different approach that produces results exceeding prior work by orders of magnitude in complexity. Rather than improving the language model itself, the proposed methodology engineers the context in which the model operates. A custom skill for Claude Code [6] was constructed to encode FPGA design expertise, enforce a simulation-first verification workflow, and provide access to live documentation through the Context7 Model Context Protocol (MCP). Using this skill, six complete RISC-V [1] processor designs were generated in a single session, verified in simulation, and validated on physical hardware. The most complex design — a 64-bit RV64IFD processor with a full IEEE 754 double-precision FPU implementing 68 instructions across 8 execution units in a 5-stage pipeline — consumes 12,969 LUTs and 18 DSP48E1 slices on a Xilinx Artix-7, with dual-language output in both Verilog and VHDL. To the best of our knowledge, this represents the most complex processor design generated by a large language model to date, exceeding prior work by orders of magnitude in instruction count, datapath width, and functional unit complexity.

The contributions of this paper are threefold. First, a skill architecture is presented that merges community-contributed FPGA design knowledge with simulation-driven verification, live documentation retrieval, and synthesis tool version awareness. Second, a three-level testbench hierarchy is described that adapts test-driven development principles to hardware verification, catching 10 floating-point bugs and driving a 48% reduction in resource utilization. Third, quantitative results are reported for six processor variants targeting the Xilinx Artix-7 XC7A35T [4], including a 64-bit RV64IFD processor with a complete IEEE 754 [2] double-precision floating-point unit consuming 12,969 LUTs.

II. Methodology

A. Custom Skill Architecture

The skill-based approach begins with the observation that LLM performance on domain-specific tasks depends critically on the context provided at generation time. Claude Code supports user-defined skills: structured context documents that prime the model with domain expertise, coding conventions, and workflow constraints before any code generation occurs. The custom FPGA skill used in this work was constructed by merging two existing community-contributed skills and augmenting them with three additional capabilities.

The first source skill focused on RTL design patterns: clock-domain crossing synchronizers, finite state machine (FSM) templates, block RAM (BRAM) inference rules, pipeline design heuristics, and DSP48E1 utilization strategies for Xilinx 7-series devices [4]. The second source skill centered on the Vivado [5] synthesis workflow: project creation via TCL scripting, synthesis and implementation settings, timing constraint idioms, and bitstream generation procedures. Neither skill was complete in isolation. A skill that encodes pipeline design patterns but lacks verification infrastructure produces code that may synthesize but cannot be validated. Conversely, a skill that automates the Vivado workflow but has no opinion on RTL structure will accept structurally deficient designs without objection.

The merged skill was augmented with three capabilities. First, a simulation-first TDD mandate was embedded as a hard constraint: the skill requires that every RTL module be accompanied by a self-checking testbench with explicit pass/fail assertions before synthesis is permitted. Second, Context7 MCP integration was added to enable live documentation retrieval. Rather than relying on the model's training data for RISC-V instruction encodings [1], IEEE 754 special-case rules [2], or Vivado TCL syntax [5], the skill queries authoritative sources at generation time. Third, Vivado version awareness was implemented to adapt generated code to the user's specific synthesis tool version. The skill detects the installed Vivado version and adjusts its output accordingly, avoiding incompatible TCL syntax, unsupported SystemVerilog constructs, and inappropriate device primitive instantiations.

The skill includes three reference files that ground every generation: `patterns.md` encodes structural design patterns, `sharp_edges.md` catalogs known failure modes and their mitigations, and `validations.md` provides regex-based lint rules for common HDL errors such as incomplete sensitivity lists and unregistered outputs.

B. Simulation-First Verification Framework

The verification framework enforces a three-level testbench hierarchy adapted from software TDD principles. At the unit level, individual modules such as the ALU and each FPU execution unit are tested with known input/output vectors and IEEE 754 test constants. At the integration level, multi-module assemblies are verified by loading test programs via \$readmemh and checking register file state and program counter progression across instruction sequences. At the system level, complete processor instances execute test programs for thousands of simulation cycles, with results mapped to GPIO outputs for hardware correlation.

This hierarchy ensures that bugs are caught at the lowest possible level, where diagnosis is cheapest. Hardware bugs that escape to synthesis consume hours of place-and-route time and produce failures that are difficult to diagnose on physical devices. Catching those same bugs in simulation costs seconds. The skill makes simulation the mandatory first step, not an optional afterthought. Fig. 2 illustrates this workflow.

C. Target Platform

All designs target the Digilent Basys 3 development board [10], which features a Xilinx Artix-7 XC7A35T-1CPG236C FPGA. This device provides 20,800 LUTs, 41,600 flip-flops, 90 DSP48E1 slices, and 50 36Kb block RAMs [4]. Synthesis, implementation, and bitstream generation were performed using Xilinx Vivado 2020.2 [5]. The board operates at a native 100 MHz clock frequency. Hardware verification employed the 16-LED array available on the Basys 3 through memory-mapped GPIO.

III. Implementation

A. Processor Architecture

The generated processor family comprises three architectural tiers. The rv16 design implements a 16-bit datapath with a 3-stage pipeline supporting a 12-instruction subset of the RV32I base integer instruction set. The rv64 design scales to a 64-bit datapath with a 3-stage pipeline implementing the full 40-instruction RV64I specification. The rv64fp design, the most complex variant, implements a 5-stage pipeline (Fetch, Decode, Execute, Memory, Writeback) supporting the RV64IFD instruction set with 68 instructions.

The rv64fp pipeline implements full data forwarding to resolve read-after-write (RAW) hazards without stalls in the common case. A dedicated hazard detection unit monitors inter-stage register dependencies and inserts pipeline bubbles for load-use hazards that cannot be resolved through forwarding alone. The architecture employs a Harvard memory model with separate instruction and data BRAM, plus memory-mapped GPIO for LED output. Fig. 1 illustrates the pipeline architecture.

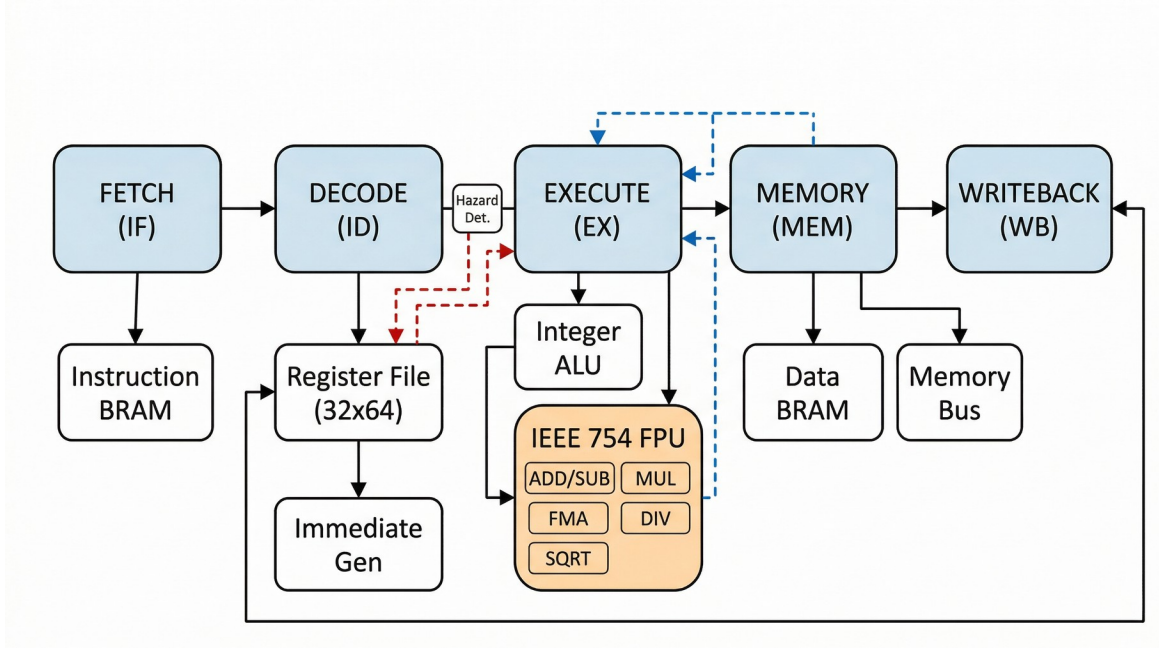


Fig. 1. Five-stage pipeline architecture of the rv64fp processor with IEEE 754 FPU.

B. IEEE 754 Floating-Point Unit

The floating-point unit implements 28 IEEE 754 double-precision operations [2] distributed across eight specialized execution units. The addition/subtraction unit employs a 3-cycle pipelined architecture with mantissa alignment shifting and leading-zero normalization. The multiplication unit uses a 3-cycle pipeline with DSP48E1 cascade [4] for full 53-by-53-bit mantissa multiplication. Fused multiply-add (FMA) operations, including FMADD, FMSUB, FNMSUB, and FNMADD, avoid intermediate rounding to achieve higher precision than sequential multiply-then-add operations. Division and square root are implemented as iterative multi-cycle units using digit-recurrence algorithms.

Additional operations include sign injection (FSGNJ, FSGNJN, FSGNJX), comparisons (FEQ, FLT, FLE) with proper NaN handling per IEEE 754, classification (FCLASS) for runtime type inspection, and format conversions (FCVT) between integer and floating-point representations in both directions. All five IEEE 754 rounding modes are supported: round to nearest even (RNE), round toward zero (RTZ), round down (RDN), round up (RUP), and round to nearest magnitude (RMM). Each execution unit independently handles special cases including NaN propagation, infinity arithmetic, signed zero, and subnormal numbers.

C. Dual-Language Output

Each processor variant was generated in both Verilog and VHDL, producing 26 RTL modules per language. The Verilog implementation totals 5,721 lines and the VHDL implementation totals 4,133 lines. Both language variants include complete Vivado TCL build scripts for project creation, synthesis, implementation, and bitstream generation, as well as shared Basys 3 constraint files (XDC format) for pin mapping and clock definition. Test programs were written in RISC-V assembly and compiled to hexadecimal memory initialization files via a Python assembler script.

IV. Results

A. Design Metrics

Table I summarizes the design metrics for all six processor variants. The designs span three orders of magnitude in complexity, from the 400-LUT rv16 educational core to the 12,969-LUT rv64fp processor with full floating-point support.

TABLE I

Design Metrics for All Processor Variants

Processor	Language	Datapath	Pipeline	ISA	Instr.	LUTs
rv16	Verilog	16-bit	3-stage	RV32I sub.	12	~400
rv64	Verilog	64-bit	3-stage	RV64I	40	3,273
rv64fp	Verilog	64-bit	5-stage	RV64IFD	68	12,969
rv16	VHDL	16-bit	3-stage	RV32I sub.	12	~400
rv64	VHDL	64-bit	3-stage	RV64I	40	3,273
rv64fp	VHDL	64-bit	5-stage	RV64IFD	68	12,969

B. Verification Results

The simulation-first verification framework identified 10 bugs in the initial FPU generation. These included rounding edge cases in the addition/subtraction unit where guard, round, and sticky bit computation failed for specific mantissa alignment scenarios; NaN propagation errors in the comparison unit where quiet NaN and signaling NaN were not distinguished correctly per IEEE 754 Section 6.2 [2]; and a sign bit inversion in the FNMSUB operation where the negation was applied to the wrong intermediate result.

Unit-level testbenches (tb_alu.v and tb_fpu_ops.v) verified each ALU and FPU operation independently. The tb_fpu_ops.v testbench spans 327 lines and exercises all 28 FPU operations with IEEE 754 test constants including positive and negative zero, infinity, quiet NaN, signaling NaN, and subnormal values. Each test case includes a descriptive label for immediate failure identification in simulation logs, along with timeout protection to detect infinite loops in the iterative division and square root units.

Integration-level testbenches (tb_rv16_core.v) loaded test programs via \$readmemh, instantiated dual-port memory models, and verified program counter progression and register file state across multi-instruction sequences. These tests detected hazard detection failures and data forwarding bugs that unit-level testing alone could not reach. System-level testbenches (tb_rv64fp_full.v) executed complete test programs for over 2,000 simulation cycles and mapped FPU computation results to LED bit positions for correlation with hardware verification.

C. Resource Optimization

The simulation-driven verification process produced a substantial reduction in resource utilization. The fp_fma module, which implements fused multiply-add operations, was reduced from 13,932 LUTs to 1,942 LUTs, an 86% reduction. The full rv64fp system decreased from approximately 25,000 LUTs to 12,969 LUTs, a 48% overall reduction. Fig. 3 illustrates the LUT utilization before and after optimization.

This optimization was not the result of explicit resource minimization techniques. Rather, it was a natural consequence of the simulation-first methodology. When testbenches clarified the exact functional requirements for each module, dead logic paths and redundant special-case handling were identified and removed. The FMA unit, for

example, contained elaborate exception handling code for cases that were already handled by upstream modules. Once simulation confirmed that these cases could not reach the FMA unit, the redundant logic was eliminated.

The final rv64fp design consumes 12,969 of the 20,800 available LUTs on the XC7A35T, representing 82.9% utilization. The design also uses 18 of the 90 available DSP48E1 slices for mantissa multiplication in the floating-point multiplier and FMA units.

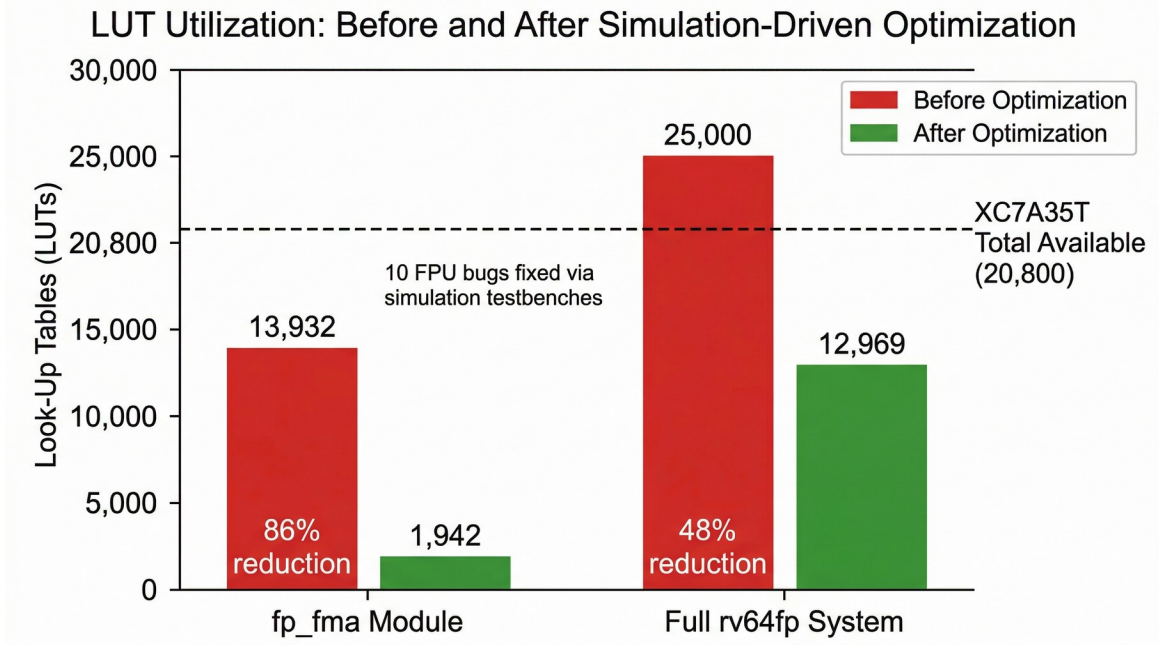


Fig. 3. LUT utilization before and after simulation-driven optimization.

D. Hardware Verification on Basys 3

All six processor variants were verified on a Digilent Basys 3 development board [10] running at the native 100 MHz clock frequency under Vivado 2020.2 [5]. Hardware verification employed the memory-mapped GPIO interface. Test programs executed on each processor and wrote results to the LED output register, where each of the 16 LED bits represented the pass/fail status of a specific test covering ALU operations, branch instructions, load/store behavior, and individual FPU operations.

This deliberately simple verification strategy was enabled by the exhaustive simulation coverage. The simulation testbenches verified functional correctness comprehensively, so the hardware test served to confirm that the synthesized netlist behaved identically to the simulation model. No discrepancies were observed between simulation and hardware behavior for any of the six designs.

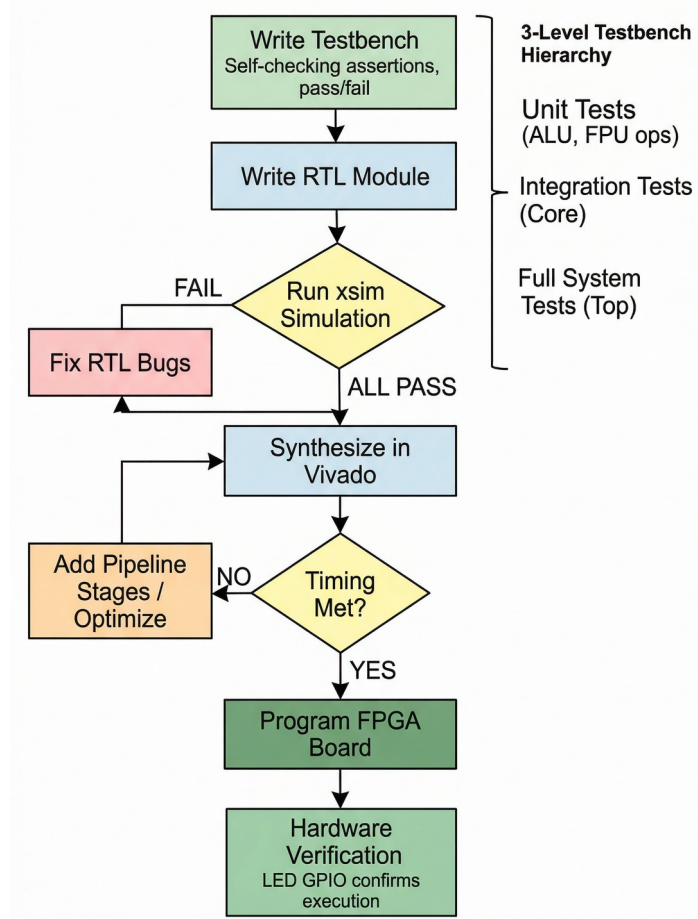


Fig. 2. Simulation-first verification workflow enforced by the custom FPGA skill.

V. Discussion

The results demonstrate that skill-based context engineering can substantially improve the quality of AI-generated HDL. Without the custom skill, the language model would produce generic Verilog with no simulation infrastructure, no version awareness, and no systematic approach to IEEE 754 special cases. The skill encoded domain expertise that the base model does not possess: patterns for clock-domain crossing, pipeline design with forwarding, BRAM inference rules, and DSP48E1 utilization strategies specific to the Xilinx 7-series architecture [4].

The simulation-first TDD mandate proved to be the most impactful component of the skill architecture. The 10 bugs caught in the initial FPU generation would have been substantially more expensive to diagnose after synthesis. More significantly, the simulation process drove a 48% reduction in LUT utilization by revealing that large portions of the generated logic were unreachable or redundant. This finding suggests that AI-generated HDL may systematically over-provision exception handling logic when not constrained by concrete test cases.

The Context7 MCP integration addressed a specific failure mode of LLM-based code generation: confident but incorrect specification of precise technical details. RISC-V instruction encodings require exact bit-field values; a single incorrect bit in the funct7 field produces a different instruction entirely [1]. IEEE 754 special-case rules are notoriously subtle: the distinction between quiet NaN and signaling NaN, the sign of zero in subtraction, and the behavior of infinity in division all have precise specifications that the model's training data may not capture accurately [2]. Live documentation retrieval eliminated this class of errors.

Several limitations of the proposed approach should be noted. First, the generated designs have not been formally verified; the verification relies entirely on simulation testbenches, which cannot exhaustively cover all possible input combinations. Second, the designs operate at the native 100 MHz clock of the Basys 3 board, but timing closure at higher frequencies has not been attempted. Third, while the 48% resource optimization is substantial, comparison with manually optimized designs by experienced engineers was not performed.

Compared to prior work in AI-assisted hardware design [7], [8], [9], [11], [12], the skill-based approach differs in both methodology and outcome. Methodologically, it engineers the generation context rather than the model or the prompting strategy. Thakur et al. [7] evaluated model capabilities on standardized module-level benchmarks, achieving at best 6.5% functional correctness on isolated components. Blocklove et al. [8] used iterative conversational refinement to produce an 8-bit accumulator-based processor — a design with no pipeline, no FPU, and a minimal instruction set. Sun et al. [11] generated a 32-bit RV32I processor without floating-point support. Zhao et al. [12] benchmarked LLM-generated modules for resource efficiency but did not attempt processor-scale designs. The proposed approach instead front-loads domain expertise into a reusable skill that transforms the model's output quality on the first generation attempt, enabling one-shot production of complete, verified designs. The resulting 64-bit RV64IFD processor with 68 instructions, a 5-stage pipeline, and 28 FPU operations represents a qualitative leap from the 8-bit accumulator that constituted the previous state of the art in AI-generated processor design.

VI. Conclusion

This paper presented a skill-based methodology for AI-assisted RISC-V processor design that produced six complete, hardware-verified processor variants in a single generation session. The custom FPGA skill, constructed by merging community-contributed design knowledge with simulation-first TDD, live documentation retrieval, and Vivado version awareness, enabled one-shot generation of designs ranging from a 12-instruction educational core to a 68-instruction RV64IFD processor with a full IEEE 754 double-precision floating-point unit.

The simulation-first verification framework identified 10 FPU bugs in the initial generation and drove a 48% reduction in LUT utilization, from approximately 25,000 to 12,969 LUTs. The most complex design consumes 82.9% of the available LUTs on the Xilinx Artix-7 XC7A35T and was verified on a Digilent Basys 3 development board at 100 MHz. The dual-language output comprises 26 RTL modules per language, totaling 5,721 lines of Verilog and 4,133 lines of VHDL.

Future work will focus on two directions. First, formal verification using property-based checking will be integrated into the skill to complement simulation-based testing. Second, the approach will be extended to more complex designs including multi-core processors, cache hierarchies, and high-speed serial interfaces to establish the boundaries of skill-based AI-assisted hardware design.

References

- [1] A. Waterman and K. Asanovic, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," RISC-V Foundation, Dec. 2019.
- [2] IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019, Jul. 2019.
- [3] D. Patterson and J. Hennessy, Computer Organization and Design RISC-V Edition. Morgan Kaufmann, 2017.
- [4] Xilinx, "7 Series FPGAs Data Sheet: Overview," DS180, 2020.
- [5] Xilinx, "Vivado Design Suite User Guide: Synthesis," UG901, 2020.
- [6] Anthropic, "Claude Code Documentation," 2026. [Online]. Available: <https://docs.anthropic.com/claude-code>
- [7] S. Thakur et al., "Benchmarking Large Language Models for Automated Verilog RTL Code Generation," in Proc. IEEE Design, Automation and Test in Europe (DATE), 2023.

- [8] J. Blocklove et al., "Chip-Chat: Challenges and Opportunities in Conversational Hardware Design," in Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD), 2023.
- [9] M. Pearce et al., "DAVE: Deriving Automatically Verilog from English," in Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD), 2020.
- [10] Digilent, "Basys 3 Reference Manual," 2021. [Online]. Available: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>
- [11] Y. Sun et al., "HDLGen-ChatGPT: RISC-V Processor VHDL and Verilog Model Generation," in Proc. Int. Workshop on Rapid System Prototyping (RSP), 2024.
- [12] T. Zhao et al., "ResBench: Benchmarking LLM-Generated FPGA Designs with Resource Awareness," in Proc. Int. Symp. Highly Efficient Accelerators and Reconfigurable Technologies (HEART), 2025.